# Task 2: Evaluating Chaotic Systems

*Quantitative Chaos Diagnostics for the Lorenz Attractor*

Tasbih Othman Mohamed Neamatalla

System and setup. The system analysed is the Lorenz system (σ = 10, ρ = 28, β = 8/3), the same chaotic system used in Assignment 1. The state was integrated with fixed-step RK4, dt = 0.01, discarding a 2000-step transient to settle onto the attractor. A scalar time series x(t) (40,001 samples) was used for all methods that require only a single observable (Parts A, B.3, C, D); the full 3-D state trajectory was used only where the ground-truth equations/Jacobian are needed (Part B.2) or where the guideline explicitly allows the original attractor points (Part E, using a denser 200,001-point / dt = 0.005 trajectory purely for finer spatial resolution).

## Part A — Phase Space Reconstruction

Time delay τ was obtained with estimate_delay_ami (max_lag=200, n_bins=32, first local minimum of the AMI curve). Embedding dimension m was obtained with estimate_dimension_fnn using that τ (max_dim=12, R_tol=10, A_tol=2, threshold=1%, theiler=50).

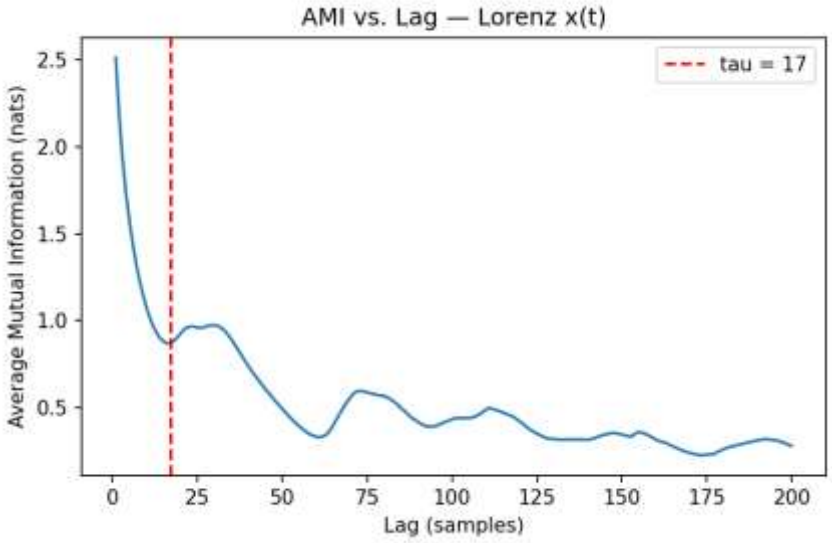


*Figure A1. AMI vs. lag for Lorenz x(t). First local minimum at τ = 17.*

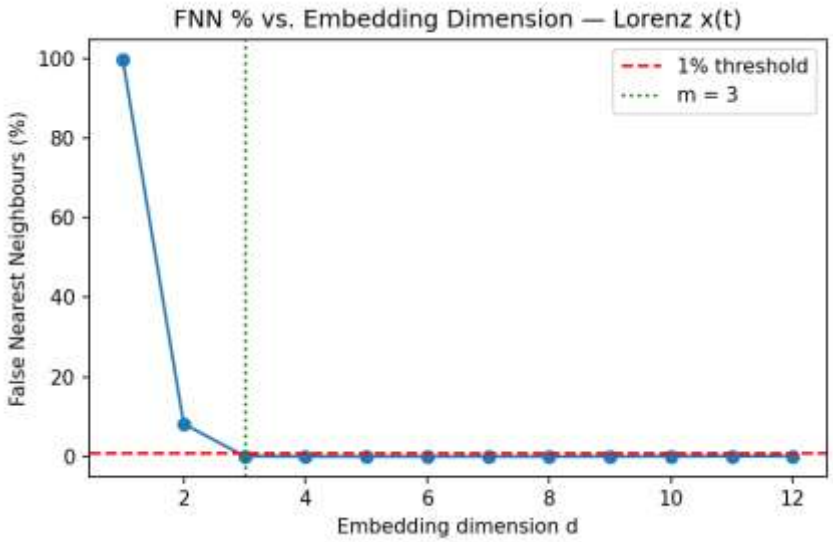


*Figure A2. FNN% vs. embedding dimension. Falls below the 1% threshold at m = 3.*

| Quantity | Result | Expected (guideline) |
|---|---|---|
| τ (time delay) | 17 samples | ~10–17 |
| m (embedding dimension) | 3 | ~3–7 (state dim. = 3) |

Both estimates land inside the expected range and m = 3 exactly matches the true state dimension of the Lorenz system — as expected, since the full flow is 3-dimensional and Takens' theorem only requires m to be large enough to unfold the attractor without ambiguity.

## Part B — Lyapunov Exponent

B.2 — ODE ground truth (Wolf GSR). Computed with lyapunov_wolf_ode using the analytic Lorenz Jacobian, manual RK4 integration (dt = 0.01), 2000 transient steps, 40,000 accumulation steps, ortho_steps = 20, log_base = 'e' (nats/time).

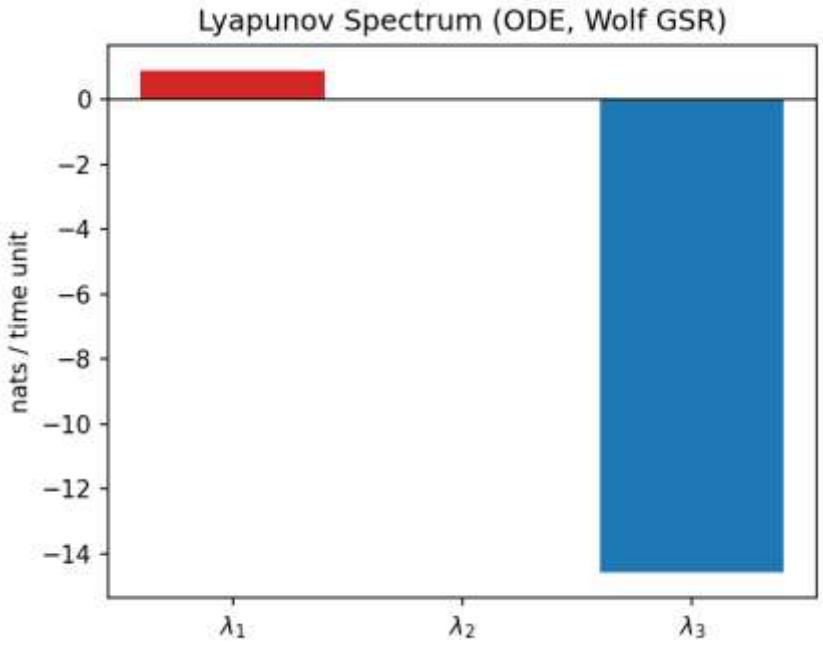

*Figure B1. Full Lyapunov spectrum from the ODE (Wolf GSR).*

| $\lambda_1$ | $\lambda_2$ | $\lambda_3$ | **Classification** |
|---|---|---|---|
| 0.9033 | −0.0005 (≈ 0) | −14.569 | Chaotic (exactly one positive exponent) |

This matches the well-known Lorenz spectrum reported in the literature (≈0.906, 0, −14.57 nats/time) closely, and confirms exactly one positive exponent → the system is chaotic, not hyperchaotic.

B.3 — Time-series estimate (Wolf tracking). Computed with wolf_mle on the scalar series x(t) using τ = 17, m = 3 from Part A (min_dist_scale=1e-3, max_dist_scale=1e-1, evolve_cap=50, max_replacements=200, angle_weight=0.3). 200 segments were successfully tracked over 99.9 time units.

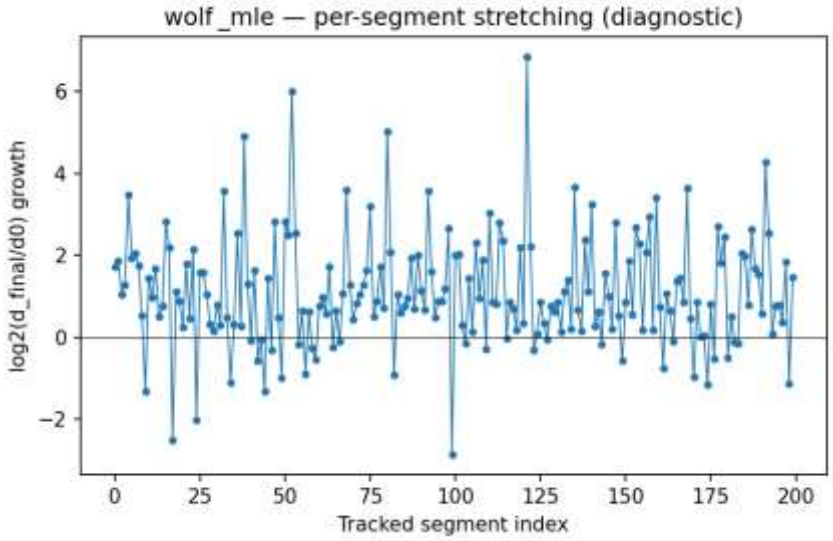


*Figure B2. Per-segment stretching (log2 growth) across the 200 tracked segments — diagnostic for tracking stability.*

| **Method** | **Value** | **Units** |
|---|---|---|
| wolf_mle (time series) | 2.2016 | bits/time |
| wolf_mle (converted) | 1.5260 | nats/time |
| lyapunov_wolf_ode (ground truth) | 0.9033 | nats/time |

The time-series estimate is roughly 69% higher than the ODE ground truth. Given the task's own guidance, the usual causes are: (i) finite series length limiting how many long, clean segments can be tracked before the Theiler/replacement machinery intervenes, (ii) reconstruction quality (m = 3 is minimal — a slightly higher m can reduce projection-induced false stretching), and (iii) sensitivity to the tracking thresholds (evolve_cap, min/max distance scale). The order of magnitude and the qualitative conclusion ($\lambda_1 > 0$ → chaos) are nonetheless consistent between both methods.

# Part C — Kolmogorov (K2) Entropy

K2 was estimated from the correlation sums computed in Part D at consecutive embedding dimensions, using C_m(r) ~ r^D2 · exp(−m·τ·dt·K2), averaged over the common scaling region of r for each (m, m+1) pair.

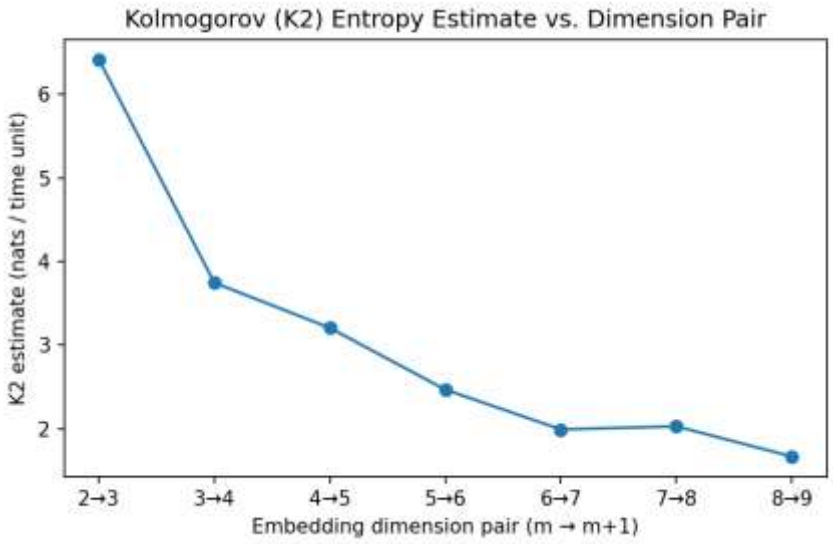


*Figure C1. K2 estimate vs. embedding-dimension pair.*

The estimate decreases steadily from ~6.4 nats/time (m: 2→3) down to ~1.6–2.0 nats/time by m: 8→9, showing a clear trend toward saturation rather than unbounded growth — the qualitative signature of low-dimensional deterministic chaos rather than noise. The saturated estimate (average of the last three pairs) is K2 ≈ 1.89 nats/time. This still sits above the Pesin-identity upper bound h_KS = $\lambda_1$ ≈ 0.903 nats/time (since K2 ≤ h_KS by definition); this is a known practical limitation of correlation-sum-based K2 estimators with finite data — they tend to overestimate entropy when the scaling region is short or under-sampled at small r, and would need substantially longer series / higher m to fully converge below the Pesin bound.

## Part D — Correlation Dimension

The Grassberger–Procaccia correlation sum C(r) was computed for embedding dimensions m = 2–9 (2000–3000-point subsamples, Theiler window = τ·m, 35 log-spaced radii spanning 0.5%–50% of the reconstructed-attractor diameter). D2 is the slope of the best-fit linear (scaling) region of ln C(r) vs. ln r.

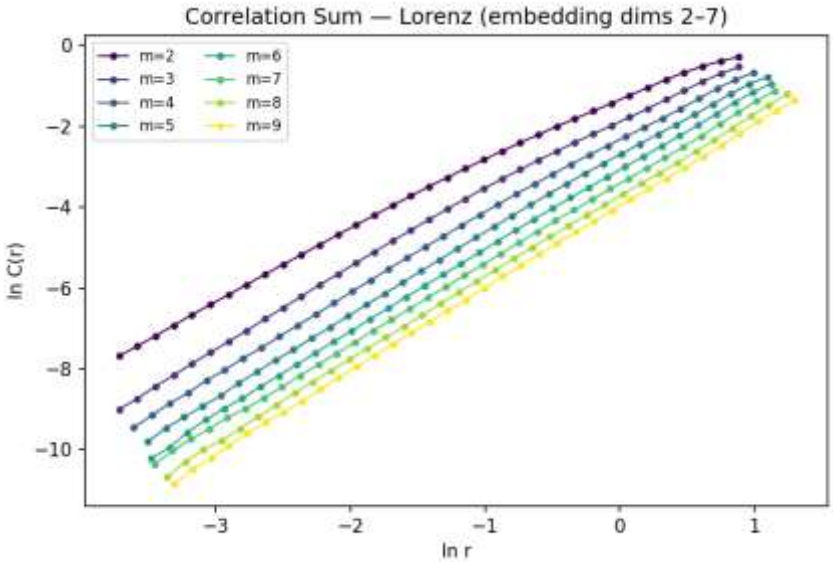


*Figure D1. ln C(r) vs. ln r for embedding dimensions m = 2–9.*

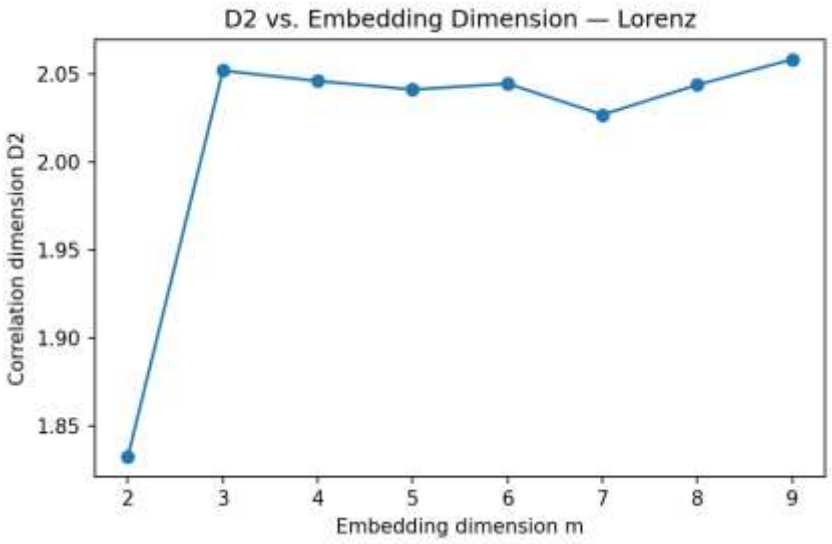


*Figure D2. Extracted D2 vs. embedding dimension m.*

D2 saturates almost immediately at m = 3 (D2 ≈ 2.05) and stays flat (2.03–2.06) through m = 9 — exactly the expected signature of a low-dimensional chaotic attractor. Saturated D2 ≈ 2.04, consistent with the well-known literature value for the Lorenz correlation dimension (≈ 2.05).

## Part E — Fractal (Box-Counting) Dimension

Box-counting was applied directly to the original 3-D Lorenz state trajectory (a denser 200,001-point run at dt = 0.005 was generated purely to give enough spatial density at fine box sizes). Box sizes ε were logarithmically spaced between half the attractor diameter and roughly twice the mean nearest-neighbour spacing; N(ε) counts occupied boxes. D0 is the slope of the best-fit linear region of ln N(ε) vs. ln(1/ε).

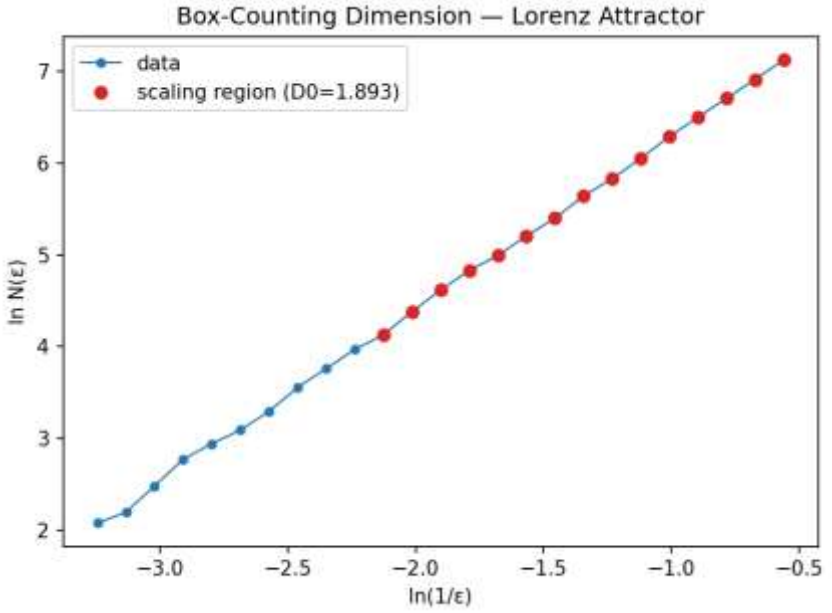


*Figure E1. ln N(ε) vs. ln(1/ε); red points mark the fitted scaling region.*

| Quantity | Value |
|---|---|

| | |
|---|---|
| D0 (box-counting) | 1.893 ($R^2 = 0.9995$) |
| D2 (correlation, saturated) | 2.043 |

D0 ≈ 1.89 is somewhat below D2 ≈ 2.04, whereas the general inequality D0 ≥ D1 ≥ D2 would predict the opposite ordering. In practice, box-counting dimension is more sensitive than the correlation sum to finite trajectory length at fine ε (empty-box undercounting), so a slight underestimate here is a known finite-sample artifact rather than a contradiction of the theory. The two values remain within about 7% of each other and both indicate a non-integer, chaotic fractal structure clearly below the embedding dimension (m = 3).

## Conclusion

Taken together, the diagnostics form a consistent picture of low-dimensional deterministic chaos in the Lorenz system: phase-space reconstruction recovers the correct state dimension (τ = 17, m = 3); the Lyapunov spectrum has exactly one positive exponent ($\lambda_1 \approx 0.903$ nats/time, classifying the system as chaotic rather than hyperchaotic), corroborated in order of magnitude by the time-series estimate; the correlation dimension saturates cleanly to D2 ≈ 2.04, a non-integer value well below the embedding dimension, as expected for a strange attractor; the box-counting dimension (D0 ≈ 1.89) is broadly consistent with D2 within the resolution limits of a finite trajectory; and the K2 entropy estimate, while not yet fully converged below the Pesin bound with the available data, trends toward saturation rather than divergence — the qualitative signature of deterministic rather than stochastic dynamics.

# Appendix — Code

psr.py and lyapunov.py are the reference implementations provided with the assignment and are used unmodified (see submission materials); they are not reproduced here. The scripts below implement the Lorenz simulation and Parts A–E of the analysis pipeline.

### lorenz_sim.py

```
"""
Lorenz system simulation (same system as Assignment 1).
Standard chaotic parameters: sigma=10, rho=28, beta=8/3.
"""
import numpy as np

SIGMA, RHO, BETA = 10.0, 28.0, 8.0/3.0

def lorenz(state, t, sigma=SIGMA, rho=RHO, beta=BETA):
    x, y, z = state
    return np.array([sigma*(y - x), x*(rho - z) - y, x*y - beta*z])

def lorenz_jac(state, t, sigma=SIGMA, rho=RHO, beta=BETA):
    x, y, z = state
    return np.array([
        [-sigma, sigma, 0.0],
        [rho - z, -1.0, -x],
        [y, x, -beta]
    ])

def rk4_integrate(x0, dt, n_steps, params=(SIGMA, RHO, BETA)):
    """Manual fixed-step RK4 integration, returns array of shape (n_steps+1, 3)."""
    n = len(x0)
    out = np.zeros((n_steps + 1, n))
    out[0] = x0
    x = x0.copy()
    t = 0.0
    for i in range(n_steps):
        k1 = lorenz(x, t, *params)
        k2 = lorenz(x + k1*dt/2, t + dt/2, *params)
        k3 = lorenz(x + k2*dt/2, t + dt/2, *params)
        k4 = lorenz(x + k3*dt, t + dt, *params)
        x = x + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)
        t += dt
        out[i+1] = x
    return out

if __name__ == "__main__":
    dt = 0.01
    transient_steps = 2000
    n_steps = 40000  # after transient

    x0 = np.array([1.0, 1.0, 1.0])
    # discard transient
    transient = rk4_integrate(x0, dt, transient_steps)
    x_settled = transient[-1]

    traj = rk4_integrate(x_settled, dt, n_steps)  # shape (n_steps+1, 3)

    np.save("lorenz_state.npy", traj)
    np.savetxt("lorenz_x.txt", traj[:, 0])
    print("Trajectory shape:", traj.shape)
    print("x range:", traj[:,0].min(), traj[:,0].max())
```

### part_A.py

```
import numpy as np
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from psr import estimate_delay_ami, AMIConfig, estimate_dimension_fnn, FNNConfig

x = np.loadtxt("lorenz_x.txt")
dt = 0.01

# --- A.2 Time delay via AMI ---
ami_cfg = AMIConfig(max_lag=200, n_bins=32, criterion="first_local_min")
tau_opt, lags, ami_vals = estimate_delay_ami(x, cfg=ami_cfg, standardize=True)
print("tau_opt =", tau_opt)

plt.figure(figsize=(6,4))
plt.plot(lags, ami_vals, lw=1.5)
plt.axvline(tau_opt, color='r', ls='--', label=f'tau = {tau_opt}')
plt.xlabel('Lag (samples)')
plt.ylabel('Average Mutual Information (nats)')
plt.title('AMI vs. Lag — Lorenz x(t)')
plt.legend()
plt.tight_layout()
plt.savefig("fig_A_ami.png", dpi=150)
plt.close()

# --- A.3 Embedding dimension via FNN ---
fnn_cfg = FNNConfig(max_dim=12, R_tol=10.0, A_tol=2.0, threshold_percent=1.0, theiler=50)
m_opt, dims, fnn_pct = estimate_dimension_fnn(x, tau=tau_opt, cfg=fnn_cfg, standardize=True)
print("m_opt =", m_opt)
print("FNN%:", fnn_pct)

plt.figure(figsize=(6,4))
plt.plot(dims, fnn_pct, 'o-', lw=1.5)
plt.axhline(1.0, color='r', ls='--', label='1% threshold')
plt.axvline(m_opt, color='g', ls=':', label=f'm = {m_opt}')
plt.xlabel('Embedding dimension d')
plt.ylabel('False Nearest Neighbours (%)')
plt.title('FNN % vs. Embedding Dimension — Lorenz x(t)')
plt.legend()
plt.tight_layout()
plt.savefig("fig_A_fnn.png", dpi=150)
plt.close()

with open("results_A.txt", "w") as f:
    f.write(f"tau_opt = {tau_opt}\n")
    f.write(f"m_opt = {m_opt}\n")
    f.write(f"FNN percentages by dimension:\n")
    for d, p in zip(dims, fnn_pct):
        f.write(f"  d={d}: {p:.3f}%\n")

np.save("tau_opt.npy", tau_opt)
np.save("m_opt.npy", m_opt)
print("Part A complete.")
```

## part_B.py

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from lyapunov import lyapunov_wolf_ode, AttractorODEConfig, WolfODEConfig, wolf_mle, WolfConfig
from lorenz_sim import lorenz, lorenz_jac, SIGMA, RHO, BETA

dt = 0.01
tau_opt = int(np.load("tau_opt.npy"))
m_opt = int(np.load("m_opt.npy"))
x = np.loadtxt("lorenz_x.txt")
```

```
# --- B.2 Ground truth spectrum from the ODE (Wolf GSR, natural log => nats/time) ---
attractor_cfg = AttractorODEConfig(
    ode=lorenz, jacobian=lorenz_jac,
    x0=np.array([1.0, 1.0, 1.0]), params=(SIGMA, RHO, BETA),
    dt=dt, transient_steps=2000, n_steps=40000,
    solver=None  # manual fixed-step RK4 (fast, matches Assignment 1 integrator)
)
wolf_ode_cfg = WolfODEConfig(ortho_steps=20, log_base='e')
spectrum = lyapunov_wolf_ode(attractor_cfg, wolf_cfg=wolf_ode_cfg)
spectrum_sorted = np.sort(spectrum)[::-1]
print("Lyapunov spectrum (nats/time), sorted descending:", spectrum_sorted)

n_positive = np.sum(spectrum_sorted > 1e-3)
classification = "chaotic" if n_positive == 1 else ("hyperchaotic" if n_positive >= 2 else "non-chaotic")

# --- B.3 LLE estimate from a scalar time series (Wolf tracking, log2 => bits/time) ---
wolf_cfg = WolfConfig(min_dist_scale=1e-3, max_dist_scale=1e-1,
                      evolve_cap=50, max_replacements=200, angle_weight=0.3)
mle_bits, debug = wolf_mle(x, dt=dt, tau=tau_opt, m=m_opt, cfg=wolf_cfg, return_debug=True)
mle_nats = mle_bits * np.log(2)  # convert bits/time -> nats/time for direct comparison

print("wolf_mle LLE (bits/time):", mle_bits)
print("wolf_mle LLE (nats/time):", mle_nats)
print("ODE lambda1 (nats/time):", spectrum_sorted[0])
print("Segments tracked:", debug["replacements"])
print("Physical time used:", debug["physical_time_used"])

# --- Plot: growth per tracked segment (diagnostic) ---
growths = [seg["growth"] for seg in debug["segments"]]
plt.figure(figsize=(6,4))
plt.plot(growths, 'o-', ms=3, lw=0.8)
plt.axhline(0, color='k', lw=0.5)
plt.xlabel('Tracked segment index')
plt.ylabel('log2(d_final/d0) growth')
plt.title('wolf\u2009_mle — per-segment stretching (diagnostic)')
plt.tight_layout()
plt.savefig("fig_B_segments.png", dpi=150)
plt.close()

# --- Bar chart comparing spectra ---
plt.figure(figsize=(5,4))
idx = np.arange(len(spectrum_sorted))
plt.bar(idx, spectrum_sorted, color=['tab:red' if v>1e-3 else ('tab:gray' if abs(v)<1e-3 else 'tab:blue')
for v in spectrum_sorted])
plt.axhline(0, color='k', lw=0.8)
plt.xticks(idx, [f'$\\lambda_{i+1}$' for i in idx])
plt.ylabel('nats / time unit')
plt.title('Lyapunov Spectrum (ODE, Wolf GSR)')
plt.tight_layout()
plt.savefig("fig_B_spectrum.png", dpi=150)
plt.close()

with open("results_B.txt", "w") as f:
    f.write(f"ODE spectrum (nats/time): {spectrum_sorted.tolist()}\n")
    f.write(f"Number of positive exponents: {n_positive} -> classification: {classification}\n")
    f.write(f"wolf_mle LLE: {mle_bits:.5f} bits/time = {mle_nats:.5f} nats/time\n")
    f.write(f"ODE lambda1: {spectrum_sorted[0]:.5f} nats/time\n")
    f.write(f"Relative difference: {abs(mle_nats-spectrum_sorted[0])/spectrum_sorted[0]*100:.2f}%\n")
    f.write(f"Segments tracked: {debug['replacements']}, physical time used:
{debug['physical_time_used']:.2f}\n")

np.save("spectrum.npy", spectrum_sorted)
np.save("mle_nats.npy", mle_nats)
print("Part B complete.")
```

## corr_dim.py

```
"""
Grassberger-Procaccia correlation sum, correlation dimension (D2), and
Kolmogorov (K2) entropy estimate, built on top of psr.reconstruct_matrix.
"""
import numpy as np
from scipy.spatial.distance import pdist, squareform
from psr import reconstruct_matrix, maybe_standardize


def correlation_sum(Y, r_values, theiler):
    """
    Compute the Grassberger-Procaccia correlation sum C(r) for each r in r_values,
    excluding pairs within a Theiler window.

    Y: (N, m) reconstructed points
    r_values: 1D array of radii
    theiler: temporal exclusion window

    Returns: C (array, same length as r_values)
    """
    N = len(Y)
    dist = squareform(pdist(Y, metric='euclidean'))
    ii, jj = np.triu_indices(N, k=1)
    valid = (jj - ii) > theiler
    d = dist[ii[valid], jj[valid]]
    n_pairs = len(d)

    C = np.array([np.sum(d < r) for r in r_values], dtype=float) / n_pairs
    return C


def scaling_region_slope(log_r, log_C, frac=0.5):
    """
    Fit a slope to the most linear central region of log_C vs log_r.
    Uses a sliding-window approach: pick the contiguous window (of length
    frac * total points) whose linear fit has the best R^2, excluding the
    flat/noisy tails.
    """
    n = len(log_r)
    win = max(4, int(n * frac))
    best_r2, best_slope, best_range = -np.inf, np.nan, (0, n)
    for start in range(0, n - win + 1):
        end = start + win
        xseg, yseg = log_r[start:end], log_C[start:end]
        if np.any(~np.isfinite(yseg)):
            continue
        A = np.vstack([xseg, np.ones_like(xseg)]).T
        slope, intercept = np.linalg.lstsq(A, yseg, rcond=None)[0]
        pred = slope * xseg + intercept
        ss_res = np.sum((yseg - pred) ** 2)
        ss_tot = np.sum((yseg - np.mean(yseg)) ** 2)
        r2 = 1 - ss_res / ss_tot if ss_tot > 0 else -np.inf
        if r2 > best_r2:
            best_r2, best_slope, best_range = r2, slope, (start, end)
    return best_slope, best_range, best_r2


def correlation_dimension_curve(x, tau, dims, n_sample=2500, n_r=30, standardize=True, theiler=None):
    """
    Compute correlation sum C(r), correlation dimension D2, for each dimension in `dims`.

    Returns dict with per-dimension: r_values, C, D2, scaling window
    and a summary array of D2 vs dims.
    """
    x = maybe_standardize(x, standardize)
```

```
    results = {}
    for m in dims:
        Y_full = reconstruct_matrix(x, tau, m)
        M = len(Y_full)
        if theiler is None:
            th = tau * m
        else:
            th = theiler

        # random subsample (kept in temporal order isn't necessary; theiler window
        # uses ORIGINAL index gaps, so subsample must be done via original indices)
        rng = np.random.default_rng(0)
        n_use = min(n_sample, M)
        idx_sub = np.sort(rng.choice(M, size=n_use, replace=False))
        Y = np.asarray(Y_full)[idx_sub]

        # theiler window applied on original-index gap: approximate using idx_sub gaps
        diam = np.max(pdist(Y)) if n_use > 1 else 1.0
        r_values = np.logspace(np.log10(0.005 * diam), np.log10(0.5 * diam), n_r)

        # Build pairwise distances with theiler exclusion based on idx_sub gaps
        N = len(Y)
        dist = squareform(pdist(Y, metric='euclidean'))
        ii, jj = np.triu_indices(N, k=1)
        gap = idx_sub[jj] - idx_sub[ii]
        valid = gap > th
        d = dist[ii[valid], jj[valid]]
        n_pairs = len(d)
        C = np.array([np.sum(d < r) for r in r_values], dtype=float) / n_pairs

        log_r, log_C = np.log(r_values), np.log(np.clip(C, 1e-12, None))
        mask = C > 0
        slope, rng_idx, r2 = scaling_region_slope(log_r[mask], log_C[mask])

        results[m] = {
            "r_values": r_values, "C": C, "D2": slope,
            "scaling_range": rng_idx, "r2": r2, "log_r": log_r, "log_C": log_C, "mask": mask
        }
    return results


def kolmogorov_k2(results, dims, tau, dt):
    """
    K2 estimate from correlation sums at consecutive embedding dimensions:
    C_m(r) ~ r^D2 * exp(-m * tau * dt * K2)
    => K2 = (1/(tau*dt)) * <ln(C_m(r)/C_{m+1}(r))>  averaged over the common scaling region.
    """
    k2_by_pair = {}
    dims = sorted(dims)
    for m1, m2 in zip(dims[:-1], dims[1:]):
        r1, C1 = results[m1]["r_values"], results[m1]["C"]
        r2v, C2 = results[m2]["r_values"], results[m2]["C"]
        # r_values are identical grid by construction only if diam similar; use m1's grid
        # interpolate C2 onto r1 grid in log space
        valid = (C1 > 0)
        logC2_interp = np.interp(np.log(r1), np.log(r2v), np.log(np.clip(C2, 1e-12, None)))
        ratio = np.log(np.clip(C1, 1e-12, None)) - logC2_interp
        # use the scaling region of m1
        s, e = results[m1]["scaling_range"]
        mask_idx = np.where(valid)[0]
        # restrict to scaling region indices intersected with valid
        region = mask_idx[(mask_idx >= s) & (mask_idx < e)] if e <= len(mask_idx) else mask_idx[s:e]
        if len(region) == 0:
            region = mask_idx
        k2_est = np.mean(ratio[region]) / (tau * dt)
        k2_by_pair[(m1, m2)] = k2_est
    return k2_by_pair
```

## part_CD.py

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from corr_dim import correlation_dimension_curve, kolmogorov_k2

dt = 0.01
tau_opt = int(np.load("tau_opt.npy"))
x = np.loadtxt("lorenz_x.txt")

dims = list(range(2, 10))  # m = 2..9, brackets known m_opt=3 and shows saturation trend
results = correlation_dimension_curve(x, tau_opt, dims, n_sample=3000, n_r=35, theiler=None)

# --- Part D: log C(r) vs log r for several m, and D2 vs m ---
plt.figure(figsize=(6.5,4.5))
colors = plt.cm.viridis(np.linspace(0, 1, len(dims)))
for m, c in zip(dims, colors):
    res = results[m]
    plt.plot(res["log_r"][res["mask"]], res["log_C"][res["mask"]], 'o-', ms=3, lw=1, color=c,
label=f'm={m}')
plt.xlabel('ln r')
plt.ylabel('ln C(r)')
plt.title('Correlation Sum \u2014 Lorenz (embedding dims 2\u20137)')
plt.legend(fontsize=8, ncol=2)
plt.tight_layout()
plt.savefig("fig_D_logCr.png", dpi=150)
plt.close()

D2_vals = [results[m]["D2"] for m in dims]
plt.figure(figsize=(6,4))
plt.plot(dims, D2_vals, 'o-', lw=1.5)
plt.xlabel('Embedding dimension m')
plt.ylabel('Correlation dimension D2')
plt.title('D2 vs. Embedding Dimension \u2014 Lorenz')
plt.tight_layout()
plt.savefig("fig_D_D2_vs_m.png", dpi=150)
plt.close()

D2_saturated = float(np.mean(D2_vals[-3:]))  # saturated value from highest dims

# --- Part C: K2 entropy from consecutive-m correlation sums ---
k2_pairs = kolmogorov_k2(results, dims, tau_opt, dt)
k2_vals = list(k2_pairs.values())
k2_estimate = float(np.mean(k2_vals[-3:]))  # saturated estimate at higher m

plt.figure(figsize=(6,4))
pair_labels = [f'{m1}\u2192{m2}' for (m1, m2) in k2_pairs.keys()]
plt.plot(pair_labels, k2_vals, 'o-', lw=1.5)
plt.xlabel('Embedding dimension pair (m \u2192 m+1)')
plt.ylabel('K2 estimate (nats / time unit)')
plt.title('Kolmogorov (K2) Entropy Estimate vs. Dimension Pair')
plt.tight_layout()
plt.savefig("fig_C_K2.png", dpi=150)
plt.close()

with open("results_C.txt", "w") as f:
    f.write(f"K2 estimates by (m->m+1) pair: {k2_pairs}\n")
    f.write(f"Saturated K2 estimate (avg of last 3 pairs): {k2_estimate:.4f} nats/time\n")

with open("results_D.txt", "w") as f:
    f.write(f"D2 by embedding dimension: {dict(zip(dims, D2_vals))}\n")
    f.write(f"Saturated D2 (avg of highest 3 dims): {D2_saturated:.4f}\n")
```

```
np.save("D2_saturated.npy", D2_saturated)
np.save("k2_estimate.npy", k2_estimate)
print("dims:", dims)
print("D2:", D2_vals)
print("D2 saturated:", D2_saturated)
print("K2 pairs:", k2_pairs)
print("K2 saturated:", k2_estimate)
print("Part C & D complete.")
```

### part_E.py

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

D2_saturated = float(np.load("D2_saturated.npy"))
traj = np.load("lorenz_state_dense.npy")  # (N, 3) densely-sampled Lorenz state trajectory

def box_count(points, eps):
    """Count occupied boxes of size eps covering the point cloud."""
    mins = points.min(axis=0)
    idx = np.floor((points - mins) / eps).astype(np.int64)
    # encode each row as a single integer key for fast uniqueness counting
    keys = idx[:, 0].astype(np.int64)
    for d in range(1, idx.shape[1]):
        keys = keys * (idx[:, d].max() + 2) + idx[:, d]
    return len(np.unique(keys))

# Use the original 3D attractor (state-space) points, as suggested by the guideline
points = traj
diam = np.max(points.max(axis=0) - points.min(axis=0))

# resolution limit ~ diam / N^(1/3) (avoid boxes smaller than typical point spacing)
N = len(points)
eps_min = diam / (N ** (1/3)) * 2
eps_max = diam / 2
eps_values = np.logspace(np.log10(eps_max), np.log10(eps_min), 25)

N_eps = np.array([box_count(points, eps) for eps in eps_values])

log_inv_eps = np.log(1.0 / eps_values)
log_N = np.log(N_eps)

# fit slope over the central scaling region (exclude saturation at small eps / large eps)
n = len(eps_values)
win = int(n * 0.6)
best_r2, best_slope, best_range = -np.inf, np.nan, (0, n)
for start in range(0, n - win + 1):
    end = start + win
    xseg, yseg = log_inv_eps[start:end], log_N[start:end]
    A = np.vstack([xseg, np.ones_like(xseg)]).T
    slope, intercept = np.linalg.lstsq(A, yseg, rcond=None)[0]
    pred = slope * xseg + intercept
    ss_res = np.sum((yseg - pred) ** 2)
    ss_tot = np.sum((yseg - np.mean(yseg)) ** 2)
    r2 = 1 - ss_res / ss_tot if ss_tot > 0 else -np.inf
    if r2 > best_r2:
        best_r2, best_slope, best_range = r2, slope, (start, end)

D0 = best_slope
s, e = best_range

plt.figure(figsize=(6,4.5))
```

```
plt.plot(log_inv_eps, log_N, 'o-', ms=4, lw=1, color='tab:blue', label='data')
plt.plot(log_inv_eps[s:e], log_N[s:e], 'o', ms=6, color='tab:red', label=f'scaling region (D0={D0:.3f})')
plt.xlabel('ln(1/\u03b5)')
plt.ylabel('ln N(\u03b5)')
plt.title('Box-Counting Dimension \u2014 Lorenz Attractor')
plt.legend()
plt.tight_layout()
plt.savefig("fig_E_boxcount.png", dpi=150)
plt.close()

with open("results_E.txt", "w") as f:
    f.write(f"D0 (box-counting dimension) = {D0:.4f}  (R^2={best_r2:.4f})\n")
    f.write(f"D2 (correlation dimension, saturated) = {D2_saturated:.4f}\n")
    f.write(f"D0 >= D2 check: {D0 >= D2_saturated}\n")

np.save("D0.npy", D0)
print("D0 =", D0, "R2 =", best_r2)
print("Part E complete.")
```